COMPUTER SCIENCE CLASSROOM TEACHING PACK

Form Two – Pages 215–223

Purpose: One classroom document containing the Teaching Summary, Teaching Notes and Teacher Presentation Script. The language is kept simple for beginner students.

How to use: First read the Teaching Summary, use the Teaching Notes when preparing, then use the Presentation Script directly while teaching.

PART 1: TEACHING SUMMARY

TEACHING SUMMARY
Computer Science – Form Two
Pages 215–223
These notes are kept very simple for beginner students. Use the Easy explanation after each point when teaching.
Main topics: Arrays, strings, file handling, debugging, errors and debugging tools.

Page 215 – Arrays and Strings
1. Array
An array is a group of values stored under one name.
Easy explanation: An array is like a row of boxes. Each box keeps one value. Each box has a number called an index.
Index: 0 1 2 3 4
Value: 1 2 3 4 5
Remember: In C, array counting starts from 0.
Example

```
#include
int main() {
int numbers[5] = {1,2,3,4,5};
for (int i=0; i<5; i++) {
printf("%d ", numbers[i]);
}
return 0;
}
```

Easy explanation: The loop moves from the first value to the last value.
2. Print in reverse

```
for (int i=4; i>=0; i--) {
printf("%d ", numbers[i]);
}
```

Easy explanation: Start from the last position and move backwards.
Output: 5 4 3 2 1
3. String
A string is text, such as a word or a name.
Easy explanation: COMPUTER is a string because it is a group of characters.

```
char word[20] = "COMPUTER";
```

4. Counting characters
A C string ends with a special character called \0. We can count characters until we reach it.

```
int count = 0;
while (word[count] != '\0') {
count++;
}
```

Easy explanation: The program counts one character at a time until it reaches the end.

Page 216 – Simple String Ideas
Full name
scanf with %s normally reads one word. A full name may contain spaces.
Easy explanation: John is one word. John Peter contains two words.
printf() and puts()

printf() displays formatted output. puts() mainly displays a string.
printf("Age = %d", age);
puts("Hello students");
Easy explanation: printf() is more flexible. puts() is simple for displaying text.
Another example
int age = 16;
printf("My age is %d", age);
puts("Welcome to Computer Science");
Easy explanation: The first line prints a value with text. puts() simply prints text.

Pages 217–219 – File Handling
1. What is file handling?
File handling means using a C program to open, read, write and close a file.
Easy explanation: A file is like a notebook. We can put information in it and use it later.
2. Main file functions
Function
Simple meaning
fopen()
Open a file
fprintf()
Write data to a file
fscanf()
Read data from a file
fclose()
Close a file
3. File modes
Mode
Simple meaning
r
Read
w
Write / replace old content
a
Add at the end
r+
Read and write
w+
Write and read
a+
Add and read
4. Writing to a file
FILE *file;
file = fopen("students.txt", "w");
fprintf(file, "John\n");
fprintf(file, "Peter\n");
fclose(file);
Easy explanation: Open the file, write the names, then close the file.
5. Reading from a file
FILE *file;
char word[50];
file = fopen("data.txt", "r");

fscanf(file, "%s", word);
printf("%s", word);
fclose(file);
Easy explanation: Open the file, read the information, display it, then close the file.
Easy memory: r = read | w = write/replace | a = add at the end

Page 220 – Debugging
1. What is debugging?
Debugging is finding and fixing errors in a program.
Easy explanation: When a program does not work correctly, we look for the problem and correct it.
2. Compilation / syntax error
This happens when the code breaks the rules of C.
printf("Hello")
Easy explanation: The semicolon is missing.
printf("Hello");
3. Logical error
The program runs but gives the wrong answer.
int a=5;
int b=2;
int result=a-b;
Easy explanation: If we want addition, use a+b. The program can run, but the answer is wrong.
4. Runtime error
A runtime error happens while the program is running.
int a=10;
int b=0;
int result=a/b;
Easy explanation: The program tries to divide by zero.

Page 221 – Three Important Errors
Error
Easy meaning
Example
Syntax
Code is written wrongly.
Missing ;
Logical
Program runs but answer is wrong.
Use - instead of +
Runtime
Problem happens while running.
Divide by zero
Easy memory: Syntax = writing mistake. Logic = wrong calculation.
Runtime = problem while running.
Example of a logical error
int marks=50;
int bonus=20;
int total=marks-bonus;
Easy explanation: If the bonus should be added, use + instead of -.
Preventing division by zero
if (b != 0) {

```
result = a / b;
} else {
printf("Cannot divide by zero");
}
```
Easy explanation: Check the number before dividing.

Page 222 – Simple Debugging Methods
1. Read the error message
Read the compiler message carefully. It gives a clue about the problem.
Easy explanation: If it says a semicolon is expected, check for a missing ;.
2. Use printf() to check values
```
int a=5;
int b=2;
int result=a+b;
printf("a=%d\n",a);
printf("b=%d\n",b);
printf("result=%d\n",result);
```
Easy explanation: This lets us see the values inside the program.
3. Fix the program step by step
Write code
↓
Compile
↓
Find error
↓
Fix error
↓
Compile again
Easy explanation: Find the problem, fix it, then test again.
4. Debugger
A debugger is a tool that helps a programmer find problems in a program.
Easy explanation: It helps us stop the program and see what is happening.

Page 223 – Debugging Tools and Exercise 4.10
1. Step through code
Run the program one line at a time.
Easy explanation: This helps you see what each line is doing.
2. Breakpoint
A breakpoint tells the debugger to stop at a chosen point.
Easy explanation: Think of a breakpoint as a STOP sign inside the program.
3. Watch variables
The debugger can show the current values of variables.
```
a = 5
b = 2
result = 7
```
Easy explanation: If a value is not correct, look for the mistake.
Exercise 4.10 – Question 1
If printf() is missing a semicolon, add it.
```
printf("Hello, World!\n");
```
Easy explanation: This is a syntax error.
Question 2
If the program should add numbers but uses minus, change - to +.

sum = num1 + num2;
Easy explanation: This is a logical error.
Question 3
If the program divides by zero, check the second number before dividing.

```
if (num2 != 0) {
result = num1 / num2;
} else {
printf("Cannot divide by zero.");
}
```

Easy explanation: This prevents the runtime problem.
Final memory: Array = group of values. File = stores information.
Debugging = finding and fixing errors. Breakpoint = stop at a chosen point.

PART 2: TEACHING NOTES

TEACHING SUMMARY
Computer Science – Form Two
Pages 215–223
These notes are kept very simple for beginner students. Use the Easy explanation after each point when teaching.
Main topics: Arrays, strings, file handling, debugging, errors and debugging tools.

Page 215 – Arrays and Strings
1. Array
An array is a group of values stored under one name.
Easy explanation: An array is like a row of boxes. Each box keeps one value. Each box has a number called an index.
Index: 0 1 2 3 4
Value: 1 2 3 4 5
Remember: In C, array counting starts from 0.
Example

```
#include
int main() {
int numbers[5] = {1,2,3,4,5};
for (int i=0; i<5; i++) {
printf("%d ", numbers[i]);
}
return 0;
}
```

Easy explanation: The loop moves from the first value to the last value.
2. Print in reverse

```
for (int i=4; i>=0; i--) {
printf("%d ", numbers[i]);
}
```

Easy explanation: Start from the last position and move backwards.
Output: 5 4 3 2 1
3. String
A string is text, such as a word or a name.
Easy explanation: COMPUTER is a string because it is a group of characters.

```
char word[20] = "COMPUTER";
```

4. Counting characters
A C string ends with a special character called \0. We can count characters until we reach it.

```
int count = 0;
while (word[count] != '\0') {
count++;
}
```

Easy explanation: The program counts one character at a time until it reaches the end.

Page 216 – Simple String Ideas
Full name
scanf with %s normally reads one word. A full name may contain spaces.
Easy explanation: John is one word. John Peter contains two words.
printf() and puts()

printf() displays formatted output. puts() mainly displays a string.
printf("Age = %d", age);
puts("Hello students");
Easy explanation: printf() is more flexible. puts() is simple for displaying text.
Another example
int age = 16;
printf("My age is %d", age);
puts("Welcome to Computer Science");
Easy explanation: The first line prints a value with text. puts() simply prints text.

Pages 217–219 – File Handling
1. What is file handling?
File handling means using a C program to open, read, write and close a file.
Easy explanation: A file is like a notebook. We can put information in it and use it later.
2. Main file functions
Function
Simple meaning
fopen()
Open a file
fprintf()
Write data to a file
fscanf()
Read data from a file
fclose()
Close a file
3. File modes
Mode
Simple meaning
r
Read
w
Write / replace old content
a
Add at the end
r+
Read and write
w+
Write and read
a+
Add and read
4. Writing to a file
FILE *file;
file = fopen("students.txt", "w");
fprintf(file, "John\n");
fprintf(file, "Peter\n");
fclose(file);
Easy explanation: Open the file, write the names, then close the file.
5. Reading from a file
FILE *file;
char word[50];
file = fopen("data.txt", "r");

fscanf(file, "%s", word);
printf("%s", word);
fclose(file);
Easy explanation: Open the file, read the information, display it, then close the file.
Easy memory: r = read | w = write/replace | a = add at the end

Page 220 – Debugging
1. What is debugging?
Debugging is finding and fixing errors in a program.
Easy explanation: When a program does not work correctly, we look for the problem and correct it.
2. Compilation / syntax error
This happens when the code breaks the rules of C.
printf("Hello")
Easy explanation: The semicolon is missing.
printf("Hello");
3. Logical error
The program runs but gives the wrong answer.
int a=5;
int b=2;
int result=a-b;
Easy explanation: If we want addition, use a+b. The program can run, but the answer is wrong.
4. Runtime error
A runtime error happens while the program is running.
int a=10;
int b=0;
int result=a/b;
Easy explanation: The program tries to divide by zero.

Page 221 – Three Important Errors
Error
Easy meaning
Example
Syntax
Code is written wrongly.
Missing ;
Logical
Program runs but answer is wrong.
Use - instead of +
Runtime
Problem happens while running.
Divide by zero
Easy memory: Syntax = writing mistake. Logic = wrong calculation.
Runtime = problem while running.
Example of a logical error
int marks=50;
int bonus=20;
int total=marks-bonus;
Easy explanation: If the bonus should be added, use + instead of -.
Preventing division by zero
if (b != 0) {

result = a / b;
} else {
printf("Cannot divide by zero");
}
Easy explanation: Check the number before dividing.

Page 222 – Simple Debugging Methods
1. Read the error message
Read the compiler message carefully. It gives a clue about the problem.
Easy explanation: If it says a semicolon is expected, check for a missing ;.
2. Use printf() to check values
int a=5;
int b=2;
int result=a+b;
printf("a=%d\n",a);
printf("b=%d\n",b);
printf("result=%d\n",result);
Easy explanation: This lets us see the values inside the program.
3. Fix the program step by step
Write code
↓
Compile
↓
Find error
↓
Fix error
↓
Compile again
Easy explanation: Find the problem, fix it, then test again.
4. Debugger
A debugger is a tool that helps a programmer find problems in a program.
Easy explanation: It helps us stop the program and see what is happening.

Page 223 – Debugging Tools and Exercise 4.10
1. Step through code
Run the program one line at a time.
Easy explanation: This helps you see what each line is doing.
2. Breakpoint
A breakpoint tells the debugger to stop at a chosen point.
Easy explanation: Think of a breakpoint as a STOP sign inside the program.
3. Watch variables
The debugger can show the current values of variables.
a = 5
b = 2
result = 7
Easy explanation: If a value is not correct, look for the mistake.
Exercise 4.10 – Question 1
If printf() is missing a semicolon, add it.
printf("Hello, World!\n");
Easy explanation: This is a syntax error.
Question 2
If the program should add numbers but uses minus, change - to +.

sum = num1 + num2;
Easy explanation: This is a logical error.
Question 3
If the program divides by zero, check the second number before dividing.

```
if (num2 != 0) {
result = num1 / num2;
} else {
printf("Cannot divide by zero.");
}
```

Easy explanation: This prevents the runtime problem.
Final memory: Array = group of values. File = stores information.
Debugging = finding and fixing errors. Breakpoint = stop at a chosen point.

PART 3: TEACHER PRESENTATION SCRIPT

TEACHER PRESENTATION SCRIPT
Computer Science – Form Two
Pages 215–223
This script is written as simple words that a teacher can say directly in class. It is designed for beginner students.
Use the questions to involve students. Do not rush. After each example, ask one or two students to explain what they see.

PAGE 215 – ARRAYS AND STRINGS
1. Introduction to an Array
Teacher says: Today we are learning about an array. An array is simply a group of values kept under one name.
Teacher says: Think about a row of boxes. Each box keeps one value. The box has a number called an index.
Index: 0 1 2 3 4
Value: 1 2 3 4 5
Teacher says: Please notice something important. In C, we start counting from zero. We do not start from one.
Ask students: If I have five values, what are the indexes?
Expected answer: 0, 1, 2, 3 and 4.
2. Printing an Array

```
int numbers[5] = {1,2,3,4,5};
for (int i=0; i<5; i++) {
printf("%d ", numbers[i]);
}
```

Teacher says: The loop starts at zero and moves forward until it reaches the last value.
Ask students: What will this program print?
Answer: 1 2 3 4 5.
3. Printing in Reverse

```
for (int i=4; i>=0; i--) {
printf("%d ", numbers[i]);
}
```

Teacher says: Here we start from the last position and move backwards.
Ask students: What will be printed?
Answer: 5 4 3 2 1.
4. String
Teacher says: A string is simply text. A name or a word can be a string.

```
char word[20] = "COMPUTER";
```

Teacher says: COMPUTER is made of many characters, so we can store it as a string.
5. Counting Characters
Teacher says: A C string ends with a special character called zero backslash, written as backslash zero.

```
int count = 0;
while (word[count] != '\0') {
count++;
}
```

Teacher says: The program checks one character at a time. When it reaches the end, it stops counting.

Teacher says: For example, CAT has three characters.

PAGE 216 – SIMPLE STRING IDEAS
1. Reading a Name
Teacher says: Now let us look at names. A name can have more than one word.
Teacher says: For example, John is one word, but John Peter has two words because there is a space.
Teacher says: When we use percent s with scanf, it normally reads one word. So students must remember that a space can separate words.
2. printf()
```
int age = 16;
printf("My age is %d", age);
```
Teacher says: printf is used to display information. It can display text and values such as numbers.
3. puts()
```
puts("Welcome to Computer Science");
```
Teacher says: puts is a simple function used mainly to display a string.
Ask students: Which one is more flexible, printf or puts?
Answer: printf.
Teacher says: So remember: printf is more flexible, while puts is simple for displaying text.

PAGES 217–219 – FILE HANDLING
1. What is File Handling?
Teacher says: Now we are going to learn about file handling.
Teacher says: File handling means using a program to work with information stored in a file.
Teacher says: Think about a notebook. We can open the notebook, write information, read information and close the notebook.
2. Four Important Functions
fopen() – open a file
fprintf() – write to a file
fscanf() – read from a file
fclose() – close a file
Teacher says: Let us use an easy memory trick: fopen means open, fprintf means write, fscanf means read, and fclose means close.
3. File Modes
r means read.
w means write or replace old content.
a means add new content at the end.
Teacher says: The most important three for beginners are r, w and a.
Ask students: If I want to read a file, which mode do I use?
Answer: r.
Ask students: If I want to add new information at the end, which mode do I use?
Answer: a.
4. Writing to a File
```
FILE *file;
file = fopen("students.txt", "w");
fprintf(file, "John\n");
fprintf(file, "Peter\n");
fclose(file);
```
Teacher says: First, we open the file. Then we write the names. Finally, we close the

file.
5. Reading from a File
FILE *file;
char word[50];
file = fopen("data.txt", "r");
fscanf(file, "%s", word);
printf("%s", word);
fclose(file);
Teacher says: Here we open the file for reading, read the information, display it, and close the file.
Tell students: r = read, w = write, a = add.

PAGE 220 – DEBUGGING
1. What is Debugging?
Teacher says: Debugging means finding and fixing errors in a program.
Teacher says: If our program has a problem, we look for the problem and correct it.
Teacher says: It is like finding a mistake in a mathematics answer and correcting the step where the mistake happened.
2. Syntax Error
printf("Hello")
Teacher says: There is a mistake here. The semicolon is missing.
printf("Hello");
Teacher says: This is a syntax error because the code was not written according to the rules of C.
3. Logical Error
int a=5;
int b=2;
int result=a-b;
Teacher says: Suppose the question asks us to add five and two. The program runs, but it gives three. The programmer used minus instead of plus.
Teacher says: That is a logical error because the program runs but gives the wrong answer.
4. Runtime Error
int a=10;
int b=0;
int result=a/b;
Teacher says: The program tries to divide ten by zero. The problem happens when the program is running. This is a runtime error.

PAGE 221 – THREE IMPORTANT ERRORS
Teacher says: Let us remember the three errors. I want you to remember them very well.
Syntax error: the code is written wrongly.
Logical error: the program runs but the answer is wrong.
Runtime error: a problem happens while the program is running.
Ask students: If I forget a semicolon, which error is it?
Answer: Syntax error.
Ask students: If I use minus instead of plus and the program gives the wrong answer, which error is it?
Answer: Logical error.
Ask students: If I divide by zero while the program is running, which error is it?
Answer: Runtime error.

Simple example
marks = 50;
bonus = 20;
total = marks - bonus;
Teacher says: If the bonus should be added, we should use plus. The program may run, but the answer will be wrong.
Teacher says: For division, we can check the number first.
if (b != 0) {
result = a / b;
} else {
printf("Cannot divide by zero");
}
Teacher says: The important idea is: check before dividing.

PAGE 222 – SIMPLE DEBUGGING METHODS
1. Read the Error Message
Teacher says: When the compiler gives an error message, do not ignore it. Read it carefully.
Teacher says: The message gives us a clue about where the problem may be.
2. Use printf()
int a=5;
int b=2;
int result=a+b;
printf("a=%d\n",a);
printf("b=%d\n",b);
printf("result=%d\n",result);
Teacher says: We use printf to see the values inside the program. This can help us find where the problem is.
3. Fix the Program Step by Step
Write code
↓
Compile
↓
Find error
↓
Fix error
↓
Compile again
Teacher says: Do not change many things at the same time. Find the problem, fix it and test again.
4. Debugger
Teacher says: A debugger is a tool that helps programmers find errors in a program.
Teacher says: It allows us to stop the program and look at what is happening.

PAGE 223 – DEBUGGING TOOLS AND EXERCISE 4.10
1. Step Through Code
Teacher says: A debugger can run the program one line at a time.
Teacher says: This helps us see what each line is doing.
2. Breakpoint
Teacher says: A breakpoint tells the debugger to stop at a particular point in the program.
Teacher says: Think of it as a STOP sign inside the program.

3. Watch Variables
a = 5
b = 2
result = 7
Teacher says: The debugger can show us the current values of variables. If a value is wrong, we can look for the mistake.
Exercise 4.10 – Question 1
Teacher says: If printf is missing a semicolon, add the semicolon.
printf("Hello, World!\n");
Teacher says: The error is a syntax error.
Question 2
Teacher says: If the program should add two numbers but uses minus, change minus to plus.
sum = num1 + num2;
Teacher says: This is a logical error because the program can run but gives the wrong answer.
Question 3
Teacher says: If the program divides by zero, check the second number before dividing.
if (num2 != 0) {
result = num1 / num2;
} else {
printf("Cannot divide by zero.");
}
Teacher says: This prevents the runtime problem.
Closing the Lesson
Teacher says: Today we learned arrays, strings, file handling and debugging.
Teacher says: The most important things to remember are: fopen opens a file, fprintf writes, fscanf reads, fclose closes, and debugging means finding and fixing errors.
Ask students: Who can tell me what a syntax error is?
Ask students: Who can tell me what debugging means?

Ask students: What does the letter r mean in file handling?
End by allowing students to answer the questions in their own words. Do not require long definitions.

QUICK CLASSROOM MEMORY

- fopen() = open a file
- fprintf() = write to a file
- fscanf() = read from a file
- fclose() = close a file
- r = read
- w = write / replace old content
- a = add at the end
- Syntax error = code is written wrongly
- Logical error = program runs but gives a wrong answer
- Runtime error = problem happens while the program is running
- Debugging = finding and fixing errors
- Breakpoint = a place where the debugger stops the program